Side Activity 6: Primary Key Changes

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, you will learn practical schema evolution skills in Oracle:

- How to **change a primary key** by **adding an attribute** (turning a simple key into a composite key).
- How to **change a composite primary key** by **removing an attribute**, and how this impacts **foreign keys** in child tables.

Part 0: Getting started

Please refer to the instructions in **Side Activity 3** to access your Oracle environment (SQL*Plus).

Part 1: Primary Key change (adding an attribute)

Key Concept: Why do we add attributes to a primary key?

Sometimes an existing primary key is no longer unique after requirements change. A common fix is to add another attribute so that the combination becomes unique (i.e., switch from a single-column key to a composite key).

Scenario

We have a course offering table. Originally, `course_id` is the primary key, meaning the table assumes each course appears only once. But in reality, a course can be offered multiple times in different terms (e.g., 2026S, 2026F).

Step 1: Create the original table (single-column PK)

```
CREATE TABLE course_offering (
  course_id   VARCHAR2(10),
  title       VARCHAR2(100),
  instructor  VARCHAR2(50),
  CONSTRAINT course_offering_pk PRIMARY KEY (course_id)
);

INSERT INTO course_offering VALUES ('CSCI3170', 'Database Systems', 'Dr. Chan');
```

Step 2: The new requirement creates a conflict

If we try to insert another offering of the same course, we get a primary key conflict:

```
INSERT INTO course_offering VALUES ('CSCI3170', 'Database Systems', 'Dr. Lee');
```

Expected error

You should see a primary key violation such as `ORA-00001: unique constraint (...) violated`.

Step 3: Add a new attribute and migrate the key

We will add a `term` column and change the primary key to `(course_id, term)`.

```
-- 1) Add the new column
ALTER TABLE course_offering ADD (term VARCHAR2(10));

-- 2) Fill existing rows (must not leave it NULL if you plan to use it in the PK)
UPDATE course_offering SET term = '2026S' WHERE course_id = 'CSCI3170';

-- 3) Drop the old primary key constraint
ALTER TABLE course_offering DROP CONSTRAINT course_offering_pk;

-- 4) Add the new composite primary key
ALTER TABLE course_offering
ADD CONSTRAINT course_offering_pk PRIMARY KEY (course_id, term);
```

Step 4: Verify it works

```
INSERT INTO course_offering VALUES ('CSCI3170', 'Database Systems', 'Dr. Lee', '2026F'
    );
SELECT * FROM course_offering ORDER BY course_id, term;
```

Part 2: Primary Key change (removing an attribute) and impacts on Foreign Keys

Key Concept: Why is removing a key attribute risky?

If you remove a column from a composite primary key, you are making the key **less specific**. This often breaks:

- the uniqueness requirement (duplicates appear),
- foreign keys in child tables (they reference the old key columns),
- application queries that used the old key.

Scenario

We track student enrollments. The enrollment record is identified by `(student_id, course_id, term)`. A child table `grade_record` references it.

Step 1: Create the parent and child tables

```
CREATE TABLE enrollment (
  student_id VARCHAR2(10),
  course_id  VARCHAR2(10),
  term       VARCHAR2(10),
  enrolled_on DATE DEFAULT SYSDATE,
  CONSTRAINT enrollment_pk PRIMARY KEY (student_id, course_id, term)
);

CREATE TABLE grade_record (
  student_id VARCHAR2(10),
  course_id  VARCHAR2(10),
  term       VARCHAR2(10),
  grade      VARCHAR2(2),
  CONSTRAINT grade_record_fk
    FOREIGN KEY (student_id, course_id, term)
    REFERENCES enrollment(student_id, course_id, term)
);
```

Step 2: Insert sample data

```
INSERT INTO enrollment VALUES ('S001', 'CSCI3170', '2026S', SYSDATE);
INSERT INTO enrollment VALUES ('S001', 'CSCI3170', '2026F', SYSDATE);

INSERT INTO grade_record VALUES ('S001', 'CSCI3170', '2026S', 'A');
INSERT INTO grade_record VALUES ('S001', 'CSCI3170', '2026F', 'B');
```

Step 3: Attempt to remove `term` from the PK (and observe the impact)

Suppose someone proposes: "term is not needed in the key". They want the primary key to become `(student_id, course_id)`.

Try dropping the PK directly:

```
ALTER TABLE enrollment DROP CONSTRAINT enrollment_pk;
```

Expected error

This will typically fail because `grade_record` still references the primary key: `ORA-02273: this unique/primary key is referenced by some foreign keys`.

Step 4: Correct migration process (drop FK, change PK, re-add FK)

To change the referenced key, you must handle the child foreign key first.

```
-- 1) Drop the child foreign key constraint
ALTER TABLE grade_record DROP CONSTRAINT grade_record_fk;

-- 2) Drop the parent primary key
ALTER TABLE enrollment DROP CONSTRAINT enrollment_pk;

-- 3) Add the new (smaller) primary key
```

```
ALTER TABLE enrollment
ADD CONSTRAINT enrollment_pk PRIMARY KEY (student_id, course_id);
```

Second conflict: duplicates may exist

After removing `term`, the rows `(S001, CSCI3170, 2026S)` and `(S001, CSCI3170, 2026F)` become duplicates under the new key `(S001, CSCI3170)`. In that case, Oracle will reject the new primary key with an error like `ORA-02437: cannot validate (...) - primary key violated`.

Step 5: Resolve duplicates (one simple approach)

For this tutorial, we will keep only one term per `(student_id, course_id)` pair. (In real design, it is usually better to **keep** term in the key or introduce a surrogate key.)

```
-- Keep only the earliest term for each (student_id, course_id)
DELETE FROM enrollment e
WHERE e.term <> (
  SELECT MIN(e2.term)
  FROM enrollment e2
  WHERE e2.student_id = e.student_id
    AND e2.course_id = e.course_id
);
```

Step 6: Re-add the foreign key (must match the new PK columns)

Now the child table must also change to reference the new key columns:

```
-- Update child table structure: remove term column or keep it as a normal attribute
ALTER TABLE grade_record DROP COLUMN term;

-- Add the new foreign key referencing the new PK
ALTER TABLE grade_record
ADD CONSTRAINT grade_record_fk
FOREIGN KEY (student_id, course_id)
REFERENCES enrollment(student_id, course_id);
```

Important (Oracle)

Oracle foreign keys support `ON DELETE CASCADE` and `ON DELETE SET NULL`, but `ON UPDATE CASCADE` is **not supported**.

Part 3: Self Challenge Exercises (not graded)

Task 1: PK change (add one attribute)

1. Create a table `event_t` with the following columns and constraints:
   - `event_id`: `VARCHAR2(10)`, initially the **only** primary key column.
   - `title`: `VARCHAR2(100)` and **NOT NULL**.
   - Name the primary key constraint exactly `event_t_pk`.

2. Insert baseline data that includes at least:
   - one row with `event_id='E001'`
   - one row with `event_id='E002'`
3. **Demonstrate the conflict**: attempt to insert a second row with `event_id='E001'` (same key). Record the error message (expect `ORA-00001`).
4. New requirement: the same event can happen every year. Evolve the schema so that:
   - a new column `event_year NUMBER(4)` exists,
   - the primary key becomes composite: `(event_id, event_year)`,
   - existing rows have a non-NULL `event_year` value (choose a year, e.g., 2026),
   - the (new) primary key constraint name is still `event_t_pk`.
5. **Verification requirements** (include the SQL you used and the output):
   - Show that you can insert **two** rows with the same `event_id='E001'` but different `event_year` values (should succeed).
   - Show that inserting a duplicate key with the same `(event_id, event_year)` fails (record `ORA-00001`).
   - Show the final rows in `event_t` sorted by `event_id` and `event_year`.
   - Show the final primary key columns and their order using the data dictionary (hint: `USER_CONS_COLUMNS`).

Task 2: PK change (remove one attribute) + foreign key impacts

1. Create a parent table `shipment_t` with:
   - columns: `order_id VARCHAR2(10)`, `warehouse_id VARCHAR2(10)`, `shipped_date DATE`, `status VARCHAR2(20)`
   - composite primary key on `(order_id, warehouse_id)`
   - PK constraint name must be `shipment_t_pk`
2. Create a child table `shipment_log_t` with:
   - columns: `log_id NUMBER` (PK), `order_id`, `warehouse_id`, `message`, `log_time DATE DEFAULT SYSDATE`
   - a foreign key referencing the parent composite key `shipment_t(order_id, warehouse_id)`
   - FK constraint name must be `shipment_log_fk`
3. Insert baseline data that satisfies all constraints and also guarantees:
   - there exists an `order_id` (e.g., `O100`) that appears in `shipment_t` with **two different** `warehouse_id` values,
   - `shipment_log_t` has at least one row for each shipment row.
4. **Attempt (expected error)**: try dropping `shipment_t_pk` while the child FK still exists. Record the error (expect `ORA-02273`).
5. **Migration goal**: change the parent primary key to only `(order_id)` (remove `warehouse_id`). Your submission must include:

- the correct order of constraint changes (drop FK first, then change PK, then re-add FK),
- an attempted `PRIMARY KEY(order_id)` that fails due to duplicates, and the recorded error message (expect `ORA-02437`).

6. Resolve the duplicate problem using **one** of the two approaches below (state which you chose):

- **Delete approach**: keep only one shipment row per `order_id` (rule: keep the smallest `warehouse_id`), and remove all other shipments for that order. Make sure child rows in `shipment_log_t` remain valid.
- **Design approach**: keep the original composite PK and write 1–2 sentences explaining why removing `warehouse_id` is invalid given your inserted data.

7. If you choose the **delete approach**, complete the migration and verify:

- parent PK is `shipment_t_pk` on `(order_id)`,
- child FK references `shipment_t(order_id)` only,
- provide final SELECT outputs for both tables.

Task 2: Clean up

Provide cleanup commands that remove all objects you created in this Side Activity. Requirements:

- Drop child tables before parent tables (foreign key dependency order).
- Your cleanup must include (if created): `v_shipment_summary`, `shipment_log_t`, `shipment_t`, `event_t`.

End of Side Activity 6.